AWS CodePipeline for Orchestration

Understand how AWS CodePipeline automates your software release process by orchestrating stages like source, build, test, and deploy. Learn to integrate testing, manual approvals, and observability to ensure high-quality, scalable deployments using AWS native and third-party tools.

Deploying software manually might work when we're just getting started, but it quickly becomes unmanageable as the application grows. AWS CodePipeline addresses this by automating the entire process, from the moment code is committed to a repository, through building and testing, all the way to deployment.

Think of CodePipeline as the project manager for the application releases. It coordinates each step, ensuring that the code passes the right checks before making its way into production. Whether working with Lambda functions or container-based apps, CodePipeline adapts to the workflow that suits the environment.

Ask

Core concepts of CodePipelines

Let's discuss the core concepts of CodePipeline to understand its working and components.

A **pipeline** is an automated workflow composed of multiple stages that build, test, and deploy code from a source repository to a production environment.

Stages

A pipeline is divided into logical *stages,* which represent a major phase of the release process. Stages are processed sequentially; the next stage only begins after all actions in the current stage have succeeded. Following are the main stages in a CodePipeline:

- The first stage is the `Source`. This is where it all begins. As soon as we commit code to a version control system like GitHub or Bitbucket, this stage detects the change and kicks off the pipeline.
- Next comes the `Build` stage. Here, tools like AWS CodeBuild compile the application, run static analysis, and conduct unit tests. If the project needs something like Jenkins instead, we can plug that in too. This stage ensures the code is functional before moving forward.
- Then we have the `Test` stage. This is where we simulate real-world usage to verify functionality. Whether it's through Lambda-based integration tests or a full-blown test framework, the goal is to catch issues early.
- Finally, there's the `Deploy` stage. CodeDeploy, CloudFormation, or even a simple S3 sync command can push our code to its target environment. Each deployment is traceable, monitored, and optionally gated by approval steps.

Actions

An action is a specific task performed within a stage. Each action is carried [illegible] an AWS service. For example, within a `Build` stage, we would have a `Build` action whose provider is AWS CodeBuild. Common actions include `source`, `build`, `test`, `deploy`, `approval`, and `invoke`.

Ask

Execution Types and Modes

Execution is the set of changes made by a pipeline. It is triggered by changes in code made manually. Each execution has its own ID. An execution can be `InProgress`, `Stopping`, `Stopped`, `Succeeded`, `Superseded`, and `Failed`.

An execution has the following types and modes:

- **Stopped Execution – Stop and Wait:** Manually halts the pipeline and waits for in-progress actions to finish before stopping completely.
- **Stopped Execution – Stop and Abandon:** Immediately halts the pipeline without waiting for ongoing actions to complete.
- **Failed Execution:** The pipeline stops due to a failure; the failed stage is unlocked and can be retried unless superseded.
- **Superseded Execution (Default Mode):** A newer execution catches up and replaces an older one at a stage boundary, halting the older execution with a `SUPERSEDED` status (cannot be retried).
- **Parallel Mode:** Multiple executions of the pipeline can run concurrently through the same stages.
- **Queued Mode:** New executions wait in line and only begin when earlier ones finish, preserving strict order.

Triggers

Triggers are events or conditions that automatically start a new pipeline execution. The most common trigger is a source change trigger, which automatically initiates the pipeline when code is pushed to a connected source repository like CodeCommit, GitHub, or Bitbucket. Pipelines can also be manually triggered through the AWS Console, CLI, or SDKs. For time-based automation, scheduled triggers can be configured using Amazon EventBridge to run pipelines at r intervals, for example, nightly builds.

Ask

Variables

Variables are key-value pairs that allow us to pass data dynamically between pipeline actions and stages. For example, we can capture the Git commit ID in the `Source` stage and pass it as a variable to the `Build` stage to be used in a build script.

Artifacts

Artifacts are the files that are passed between pipeline stages. An artifact can be either an input or an output of an action. For example, the `Source` action's output artifact is the source code, which then becomes the input artifact for the `Build` action. CodePipeline stores all artifacts in a designated S3 bucket.

Tooling and integrations

One of CodePipeline's most powerful features is its ability to work seamlessly with both AWS-native and external tools.

We can start with GitHub for our source control, move through CodeBuild for building and testing, and finish with CodeDeploy to push updates. But the flexibility doesn't end there. Maybe a development team prefers GitHub for version control or Jenkins for builds. CodePipeline supports a variety of third-party tools through prebuilt integrations and custom actions.

What's more, we can define and manage our pipeline using the AWS Console, script it with the AWS CLI, or build it into our infrastructure using the AWS SDK in our favorite language. It's as hands-on or hands-off as we want it to be.

Integration of CodePipeline with Step Functions

Ask

AWS CodePipeline has one limitation. It executes all the actions within the pipeline and can not dynamically skip steps. To understand it better, consider a

Git repository containing frontend and backend for an application. The deployment of backend and frontend is handled by a single CodePipeline.

Now, a developer makes a one-line change to the CSS in the `frontend-app` directory and pushes the commit. The CodePipeline would trigger a new execution. Since the pipeline cannot dynamically skip steps, it will proceed to run the build, test, and deploy actions for both frontend and backend applications, leading to wasted resources. Ideally, the pipeline should detect that only the `/frontend-app` directory changed and skip the stages related to the backend. To achieve this, developers must implement custom workarounds.

We can integrate CodePipeline with AWS Step Functions to dynamically execute pipeline actions. The architecture diagram below shows the infrastructure.

Integration of CodePipeline with Step Functions

Ask

This automated workflow begins whenever a developer pushes a code change to the GitHub repository, instantly triggering an AWS CodePipeline.

The pipeline pulls the latest source code, runs an initial build using AWS CodeBuild, and securely stores the resulting artifact in an Amazon S3 bucket. Upon a successful build, the pipeline hands off control to an AWS Step Functions workflow to orchestrate the next stage.

The Step Functions workflow then processes the artifact from the initial build. It iterates through each item in the artifact, using a Choice state to dynamically decide the next action, based on the service name:

- If the service is `frontend-app`, it triggers a specific CodeBuild project designed only to build and test the frontend.
- If the service is `backend-api`, it triggers a different CodeBuild project for the backend.

The entire process is marked as successful only after every invoked build has completed without any errors.

Integrating testing in Pipelines

Integrating testing into a pipeline is done by adding dedicated stages and actions that execute tests at different phases of the release process, providing automated quality gates.

Unit testing

Unit tests are the fastest and most common type of automated test. They are typically run as part of the `Build` stage. We specify the commands to run tests in the `buildspec.yml` file as shown below:

index.yaml YAML

Ask

```yaml
phases:
  build:
    commands:
      - echo Running unit tests...
```

```
        - npm test
        - echo Building the application...
        - npm run build
```

Sample buildspec.yml file

A build failure immediately stops the pipeline, preventing code with failing unit tests from being deployed.

Integration and end-to-end testing

These more comprehensive tests are run after the application has been deployed to a staging or testing environment. We add a new `Test` stage to the pipeline that occurs after the `Deploy-Staging` stage. This `Test` stage will have an action that triggers a separate AWS CodeBuild project. This CodeBuild project contains the logic to run the integration or end-to-end test suite using frameworks like Cypress or Selenium, against the live staging URL.

Ask

Integrating testing in CodePipeline using CodeBuild

If the integration tests fail, the pipeline stops, preventing a promotion to the production environment.

Adding approvals and manual gates

Automation is fantastic, but sometimes, human judgment is non-negotiable. For example, we might want a QA lead or a product owner to sign off before production deployments.

CodePipeline allows us to insert manual approval actions at any point in the pipeline. When this step is reached, the pipeline pauses and waits for someone with the right IAM permissions to either approve or reject the continuation.

Notifications are automatically sent through Amazon SNS, so the right pers pinged without delay. This adds a layer of safety and compliance that's crucial in many production environments.

Ask

Observability and alerts

Imagine a build fails, and no one notices. That's where observability comes in.

AWS CodePipeline integrates closely with Amazon CloudWatch, which lets us monitor metrics like stage duration and failure rates. We can also create custom alarms that notify us when a stage takes too long or fails entirely.

For audit trails, AWS CloudTrail records every API call, including who approved what and when. This becomes invaluable when tracing bugs or verifying compliance with change control policies.

A real-life example

Let's put it all together with a real-world scenario.

We're building a serverless web app hosted in GitHub. Every time someone pushes to the main branch, the pipeline springs into action. CodeBuild compiles the code and runs unit tests. Lambda functions kick in to perform integration testing. Once those pass, a manual approval step ensures the team lead gives the green light. Finally, CodeDeploy pushes the app to an S3 bucket for production access.

This pipeline isn't just efficient—it's transparent, auditable, and scalable.

The CodePipeline can be created using a straightforward AWS CLI command. However, it requires a JSON file to define the structure of the pipeline. Here is a simple pipeline definition for the use case example:

index.yaml YAML

Ask

```
{
  "pipeline": {
    "name": "Serverless-WebApp-Pipeline",
    "roleArn": "arn:aws:iam::123456789012:role/service-role/AWS-CodeP
    "artifactStore": {
```

```
        "type": "S3",
        "location": "my-codepipeline-artifacts-bucket"
      },
      "stages": [
        {
          "name": "Source",
          "actions": [
```

pipeline.json for use case example

Let's understand the main segments of this file:

- **Lines 2–8:** We define the name, role, and artifacts location for our pipeline, followed by the stages of the pipeline.
- **Lines 10–29:** The `Source` stage is automatically triggered by a `git push` to the `main` branch. It pulls the latest source code and passes it as an output artifact to the next stage.
- **Lines 30–48:** The `Build` stage takes the source code as input. It runs a build process that compiles the code and executes all unit tests as defined in the `buildspec.yml` file. If the build and tests succeed, it produces a new artifact.
- **Lines 49–66:** The `Integration-Test` stage takes the built artifact as input and invokes a Lambda function. The function is responsible for running automated integration tests against the built application. The pipeline only proceeds if the Lambda function reports a successful test run.
- **Lines 67–83:** The pipeline pauses at the `Approval` stage. It sends a notification via Amazon SNS to the designated team lead, who must review the changes and manually approve or reject the deployment to production.
- **Lines 84–102:** Once approved, the `Deploy-Production` stage takes the built artifact and uses CodeDeploy to push the serverless application to the production Amazon S3 bucket for hosting.

Ask

We can deploy this pipeline defined in the `pipeline.json` file using the AWS CLI command given below:

main.sh Shell

```
aws codepipeline create-pipeline --cli-input-json file://pipeline.jso
```

Ask